

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 08 · LECTURE NOTES

TLS, as the Worked Example

Every primitive so far, composed correctly.

MODULE OUTCOMES

- Explain what each handshake step establishes.
- Explain what TLS 1.3 removed and why.
- Explain what TLS does and does not protect.
- Explain mutual authentication and when to use it.
- Read a real connection and assess it.

1 The handshake

CLIENT HELLO

supported versions, cipher suites, and an ephemeral key share (Module 06 section 4)
-> establishes nothing yet; it is an offer

SERVER HELLO

the chosen suite, and its own ephemeral key share
-> both sides can now derive a shared secret.
Confidentiality, with SOMEBODY.

SERVER CERTIFICATE + SIGNATURE

the certificate chain, and a signature over the handshake transcript so far
-> authentication. This is the step that says WHO, and it is the reason Module 07 exists.

FINISHED (both sides)

a MAC over the entire transcript
-> integrity of the negotiation itself, so a downgrade or any modification is detected.

THEN application data, under an AEAD construction (Module 04), with keys derived from the exchange.

THE SIGNATURE IS OVER THE TRANSCRIPT, WHICH IS WHAT DEFEATS THE RELAY

Signing the transcript binds the server's identity to *this particular exchange* — including the specific ephemeral key shares that were actually sent by both parties.

So a relaying adversary cannot reuse a captured signature. The adversary's own exchange with the client necessarily has different key shares, so the transcript differs, so the captured signature does not verify over it — and the adversary cannot produce a fresh one without the server's private key.

Which is exactly the fix for Module 06 §1: unauthenticated key exchange gives you a shared secret with somebody, and a transcript signature is what names who. The two halves of the problem, solved by two different primitives, composed in this specific order.

And this is why 'sign the handshake' and 'sign a challenge' differ in strength — a signature not bound to the session can be replayed into a different session, which is a mistake made regularly in custom authentication protocols and is one of the strongest arguments for Module 01 §4's rule about assembling protocols.

2 What TLS 1.3 removed

What was removed	The reason
Non-ephemeral (static) key exchange	Forward secrecy is now mandatory rather than optional (Module 06 §4) — the option to omit it was itself the problem.
CBC mode, and MAC-then-encrypt	Padding oracle attacks (Module 04 §2 and §4) — a long sequence of them, each patched and then re-found in a variant.
RSA key transport	Bleichenbacher's attack and its two decades of descendants (Module 06 §3), which kept returning because the construction was fragile.
Renegotiation	A family of injection attacks in which an adversary's requests were prepended to an authenticated session.
Compression	The compression ratio leaks plaintext — an adversary who can inject guesses observes whether they compressed well, recovering secrets byte by byte.
Custom Diffie–Hellman groups	Weak and potentially backdoorable parameters (Module 06 §4), and no way for a client to assess a group it was handed.

NEGOTIABLE WEAKNESS IS ITSELF A VULNERABILITY

If a protocol is able to negotiate a weak option, then an adversary who can influence the negotiation will cause it to be selected — which turns every option retained for backwards compatibility into an active attack surface rather than a dormant one.

And this produced a full decade of downgrade attacks — version rollback, the revival of deliberately weakened export-grade ciphers long after they were abandoned, cross-protocol confusion — **all of which exploited options kept for compatibility with systems nobody was still running.**

So TLS 1.3's approach was to remove rather than deprecate, and additionally to **authenticate the negotiation itself** (§1's Finished message) so that any tampering with what was offered or chosen is detected before application data flows.

Which generalises well beyond TLS: in any system you design, a configuration option that is unsafe will eventually be selected — by an unfortunate default, by a copied Stack Overflow example, by a well-meaning compatibility fix, or by an adversary. **The only reliable remedy is for the unsafe option not to exist** (CSCE 701 Module 11 §3's defaults argument, arriving from the other direction).

3 What it protects

- **It protects data in transit between two endpoints** — confidentiality, integrity, and authentication of the server. **Three of Module 01's four properties, for one hop.**
- **It does not protect data at either endpoint. A compromised server sees plaintext, because it must** — which is precisely where end-to-end encrypted designs differ, and is the architectural question of Module 10 §4.
- **It does not hide metadata. The destination address, the connection timing, and the traffic volume are all visible to a network observer, and the requested server name was historically visible in the clear** as well (Module 01 §2's first limitation).
- **It does not authenticate the client** unless you specifically configure mutual TLS (§4) — so **authentication is one-directional by default**, and the application layer is left to identify the user.
- **And it says nothing whatsoever about authorisation** — **an authenticated connection from a party who should not be permitted to perform the requested action is still a vulnerability** (CSCE 701 Module 03). **'We use TLS' answers one question out of four, and the endpoint-compromise boundary is the one that matters most in practice**, because it is the one people genuinely misjudge when describing their system's guarantees to users.

4 Mutual TLS, and reading a connection

MUTUAL TLS AUTHENTICATES BOTH ENDS WITH CERTIFICATES

The client presents a certificate of its own, and the server validates it against a trusted issuer — so **both ends are cryptographically identified** rather than only one, and the application layer need not re-establish who the caller is.

Which makes it excellent for service-to-service authentication: no shared secrets to distribute, identity bound to a key rather than to a bearer token that can be stolen and replayed, and it composes cleanly with CSCE 701 Module 04's segmentation and Module 03's authorisation.

And it is operationally demanding. Every client needs a certificate, and every certificate needs issuance, secure distribution, rotation before expiry, and revocation when a client is decommissioned — **which is Module 07 §4's unsolved problem, now internal to your infrastructure.** A service mesh exists largely to automate exactly this.

So the honest assessment: right for service-to-service communication inside infrastructure you operate and can automate, and usually wrong for end users — where the certificate lifecycle lands on people who cannot reasonably manage it, which is CSCE 701 Module 11 §1's usability argument deciding a cryptographic question.

- **The negotiated version and cipher suite. TLS 1.3 where possible, and TLS 1.2 with an AEAD suite as the minimum** — and nothing older, which is now a configuration question rather than a compatibility one.

- **Whether the key exchange is ephemeral**, which determines forward secrecy and is visible in the suite name itself (the ECDHE prefix).
- **The certificate chain, its expiry, and its issuer** — and critically **whether the name matches what you actually asked for** rather than what the certificate offers (Module 07 §2's most-omitted check).
- **Whether your client library validates by default, and whether anything in your own codebase disables it** — which is a grep worth running across every repository you own.
- **And whether the server's TLS implementation is current**, which is a patching question (CSCE 701 Module 08) rather than a cryptographic one. **Run this against your own services: most findings turn out to be configuration and currency rather than cryptography** — which is this course's second theme, restated with evidence you gathered yourself.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
RFC 8446 — TLS 1.3 (free)	§1 and §2 in the original — and the handshake section is genuinely readable, which is unusual for an RFC.
Ristic — Bulletproof TLS and PKI	The whole module as the practical reference , including the configuration guidance for §4. Library copy.
The Illustrated TLS 1.3 Connection (free)	§1 byte by byte , annotated — the single best way to see the handshake concretely.
SSL Labs server test , and testssl.sh (free)	§4's checks, automated — run against your own services, and read the explanations rather than only the grade.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Capture a TLS 1.3 handshake** and identify each of the four steps.
- 2 **Say what each step established**, in your own words.
- 3 **Explain why a captured transcript signature cannot be replayed** by a relay.
- 4 **Contrast signing a transcript with signing a challenge**, and state which is stronger.
- 5 **For each of the six removed features**, name the attack it enabled.
- 6 **Find one unsafe configuration option** in a system you operate, and ask whether it could be deleted.
- 7 **List what an observer learns** about your TLS traffic despite the encryption.

- 8 **Test one of your own services** with a TLS scanner and read every finding.
- 9 **Classify each finding** as cryptographic, configuration, or currency.
- 10 **Decide whether mutual TLS is right** for one interface you operate, and justify it on lifecycle grounds.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the four handshake steps and what each establishes.
- 2 Why is the transcript signature the critical step?
- 3 Why is signing a challenge weaker than signing a transcript?
- 4 Name six things TLS 1.3 removed and the attack each closed.
- 5 Why is a negotiable weak option a vulnerability?
- 6 State the general design lesson and where else it applies.
- 7 Give five things TLS does not provide.
- 8 Which boundary is most often misjudged?
- 9 When is mutual TLS right, and when wrong?
- 10 What five things do you check when reading a connection?